

Perceptron & Support Vector Machines

Hrachya Asatryan

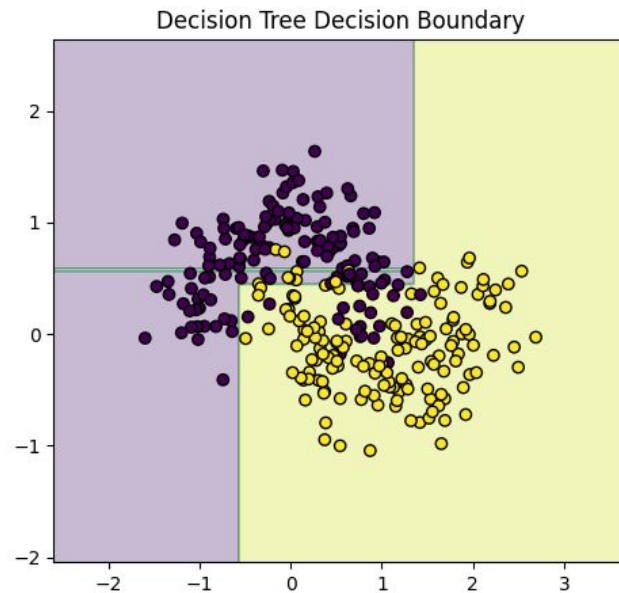
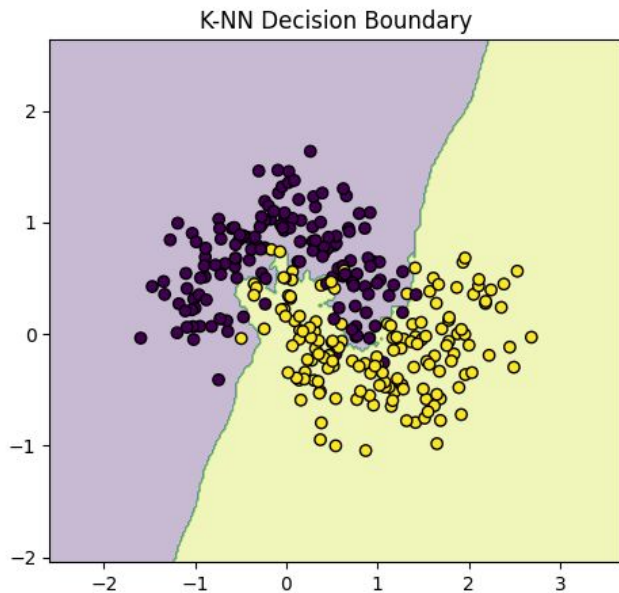
Decision space

The task of binary classification is essentially splitting the feature space into 2 parts for each class.

Examples:

- K-NN splits the feature space based on **local neighborhoods**.
- Decision Trees split the feature space based on **axis-aligned partitions**.

Illustration

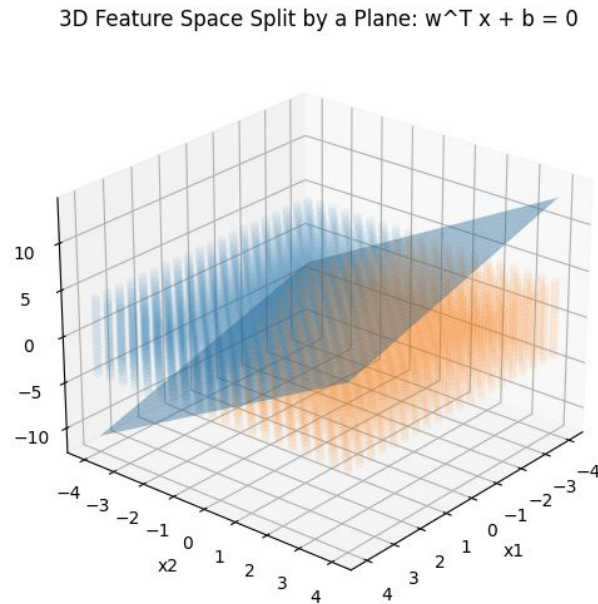
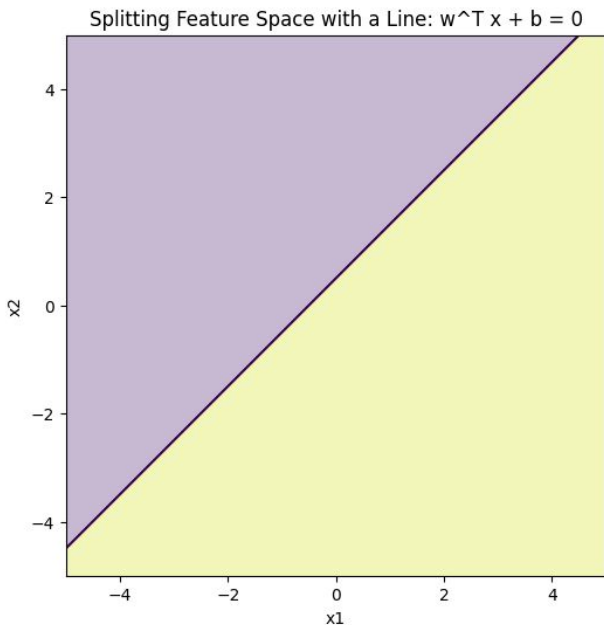


What are some other ways we can split the feature space?

We can simply draw a straight line

Linear Separators

One can choose any line (hyperplane) $\mathbf{w}^T \mathbf{x} + b$ to split the feature space into 2 parts:



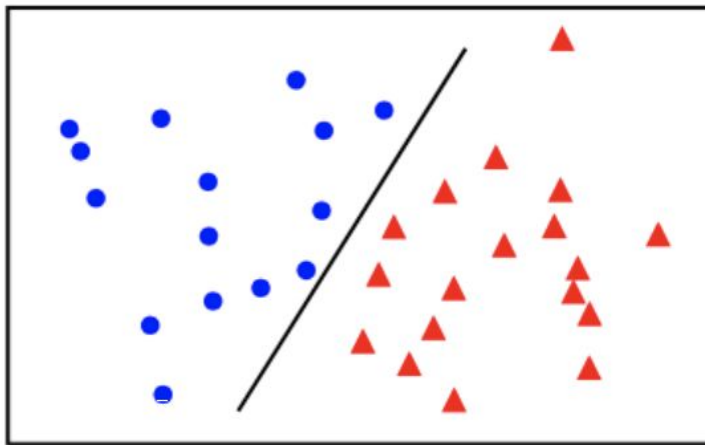
Linear Separators

We can then classify the points according to which side of the line they are. Mathematically this means

$$\mathbf{w}^T \mathbf{x} + b > 0 \quad \bullet$$

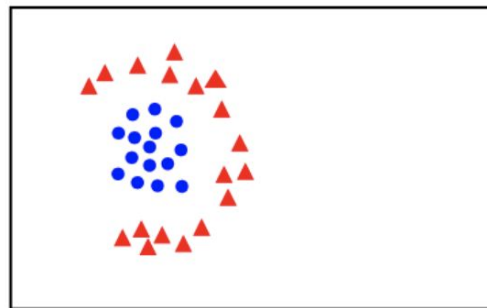
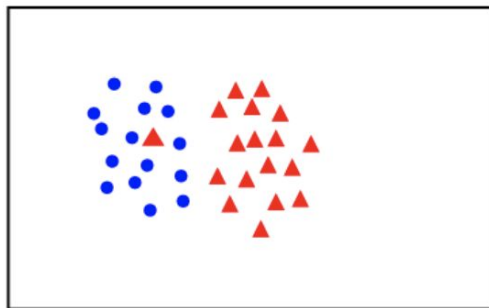
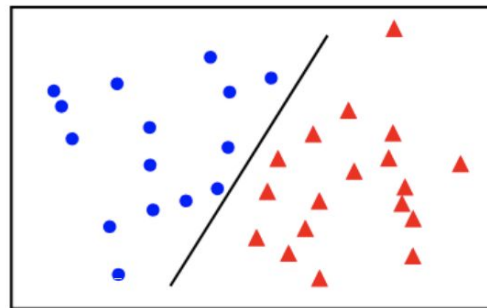
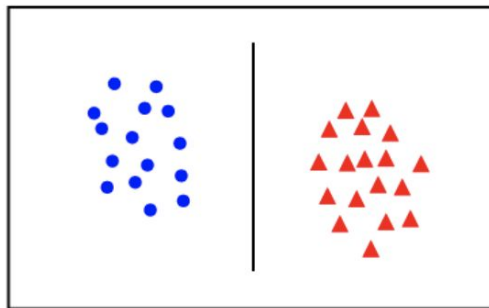
$$\mathbf{w}^T \mathbf{x} + b = 0 \quad ?$$

$$\mathbf{w}^T \mathbf{x} + b < 0 \quad \blacktriangle$$



Linear Separators

Not all datasets are linearly separable



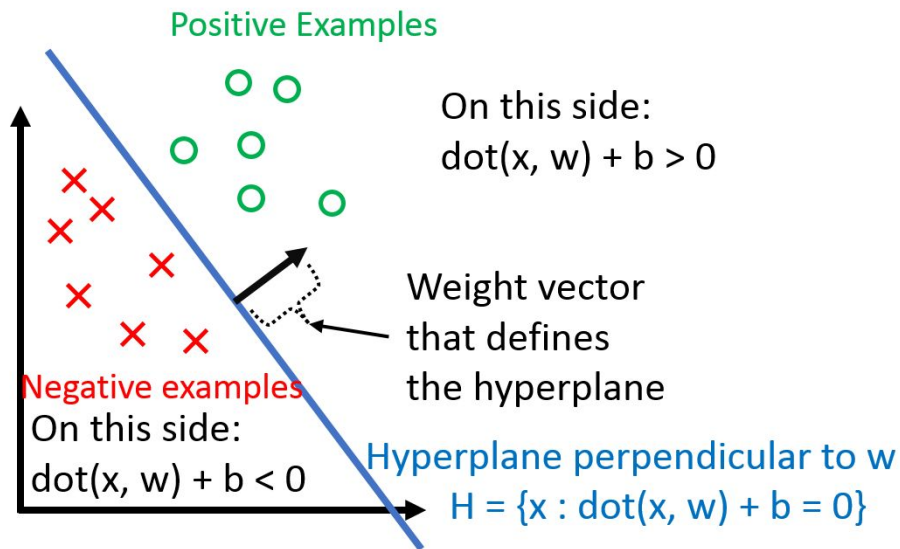
How can we find a good line to separate?

Perceptron

Perceptron Algorithm

In classical ML, the Perceptron is a linear classifier + a specific **mistake-driven training algorithm**.

$$h(x_i) = \text{sign}(\mathbf{w}^\top \mathbf{x}_i + b)$$



Perceptron Algorithm

If we denote the 2 classes as $\{-1; 1\}$, one can notice that

$$y_i(\mathbf{w}^\top \mathbf{x}_i) > 0 \iff \mathbf{x}_i \text{ is classified correctly}$$

Perceptron Learning Rule

Given training data $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$

Let $\boldsymbol{\theta} \leftarrow [0, 0, \dots, 0]$

Repeat:

Let $\boldsymbol{\Delta} \leftarrow [0, 0, \dots, 0]$

for $i = 1 \dots n$, do

if $y^{(i)} \mathbf{x}^{(i)} \boldsymbol{\theta} \leq 0$

// prediction for i^{th} instance is incorrect

$\boldsymbol{\Delta} \leftarrow \boldsymbol{\Delta} + y^{(i)} \mathbf{x}^{(i)}$

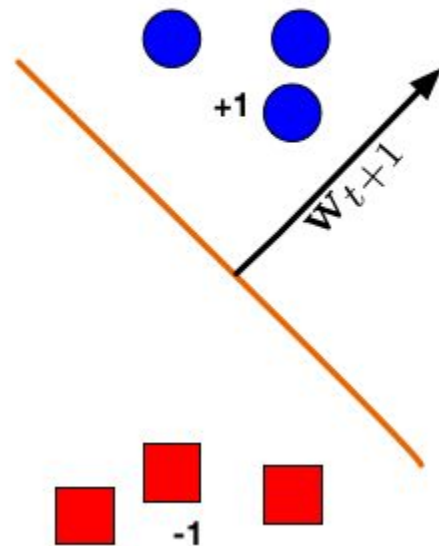
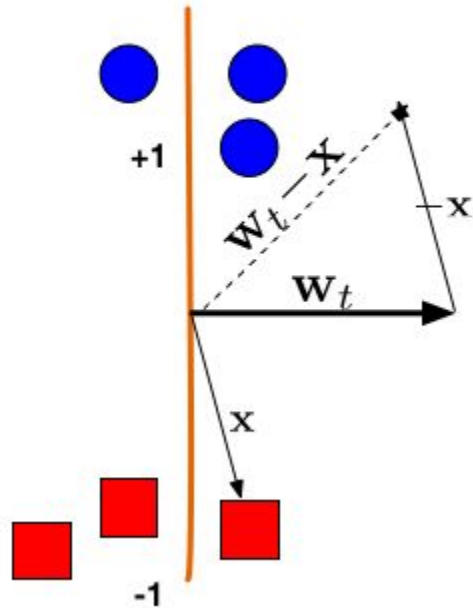
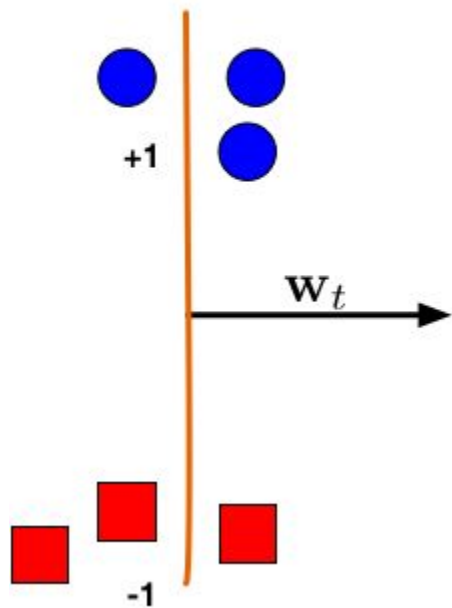
$\boldsymbol{\Delta} \leftarrow \boldsymbol{\Delta} / n$

// compute average update

$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \boldsymbol{\Delta}$

Until $\|\boldsymbol{\Delta}\|_2 < \epsilon$

Illustration



Perceptron History

In 1957, before convex optimization machinery was mainstream, the perceptron was the first algorithm that:

- Adjusted parameters iteratively
- Had a convergence theorem*
- Worked in high dimensions

It was the first attempt at learning a linear classifier.

Perceptron Convergence

Theorem: If a data set is linearly separable, the Perceptron will find a separating hyperplane in a finite number of updates. If the data is not linearly separable, it will loop forever.

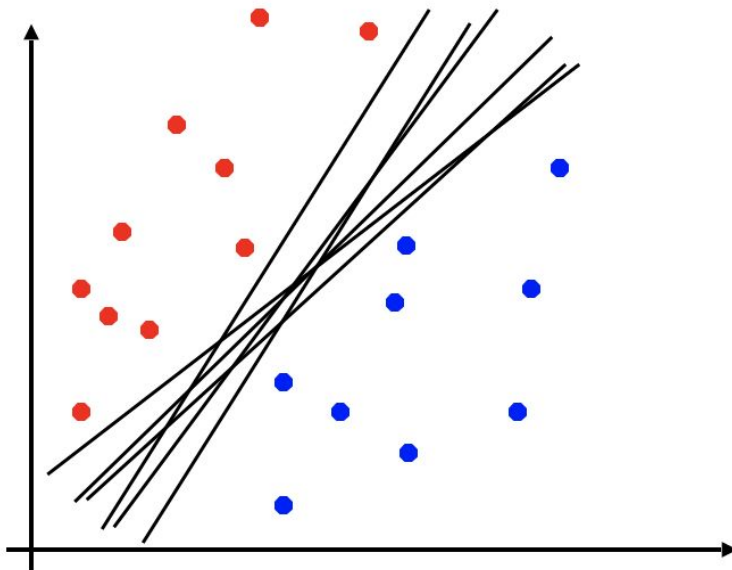
Proof:

<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecture03.html>

Support Vector Machines

Margins

There are many lines that can split the feature space into 2. How do we choose?



Problem Formulation

What do we have?

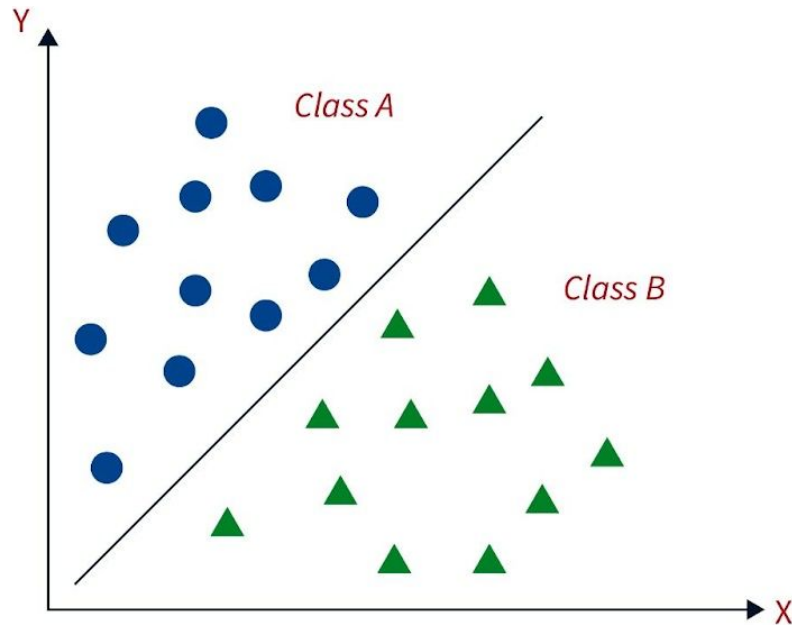
A training data set comprising $\mathbf{N}$ observations $\{\mathbf{x}_n\}$, where $n = 1, \dots, N$, together with corresponding target values $t_1, \dots, t_N$, $t_n \in \{-1, 1\}$

What is our goal?

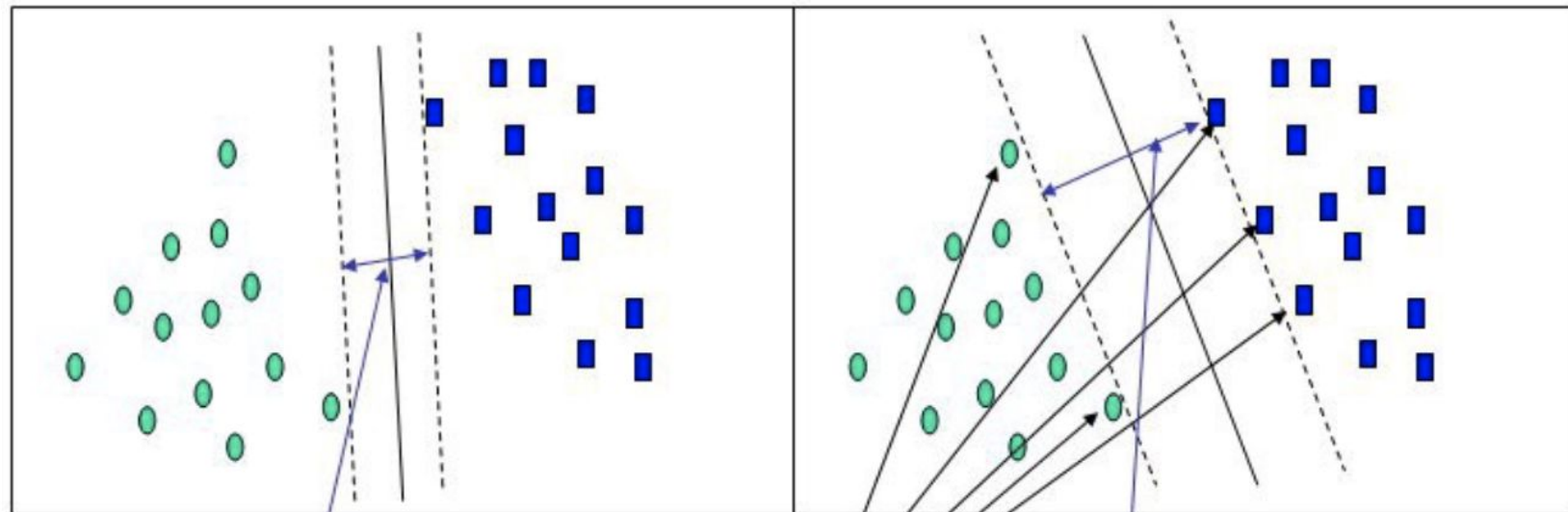
Take an input vector $\mathbf{x}$ and to assign it to one of K classes, while searching the solutions in a sparse space.

How do we do that?

We try to maximize the distance between the closest points to the decision boundary.



Margins

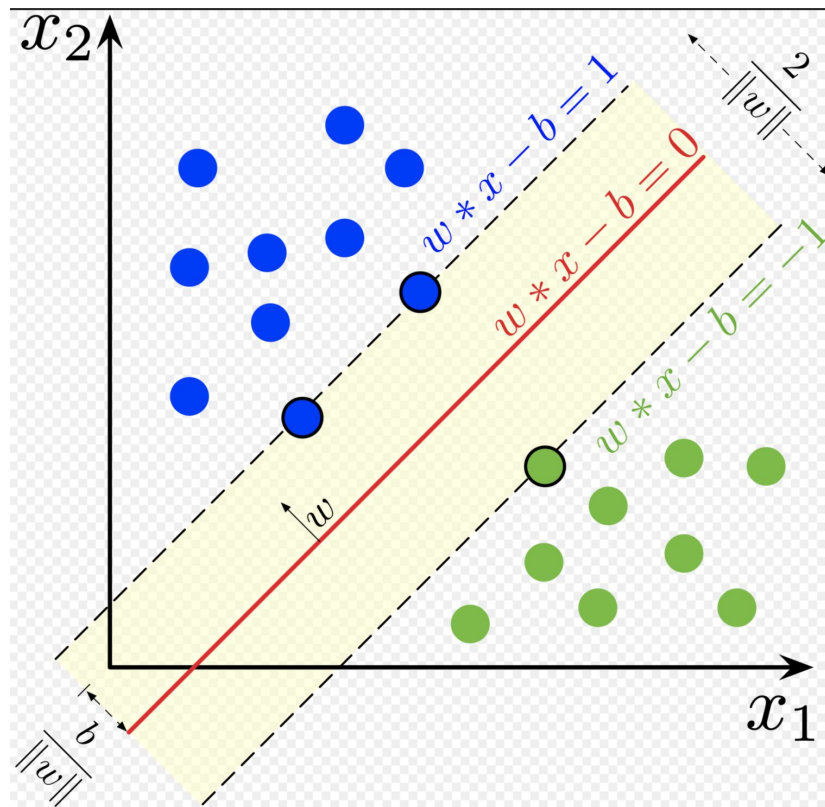


Small Margin

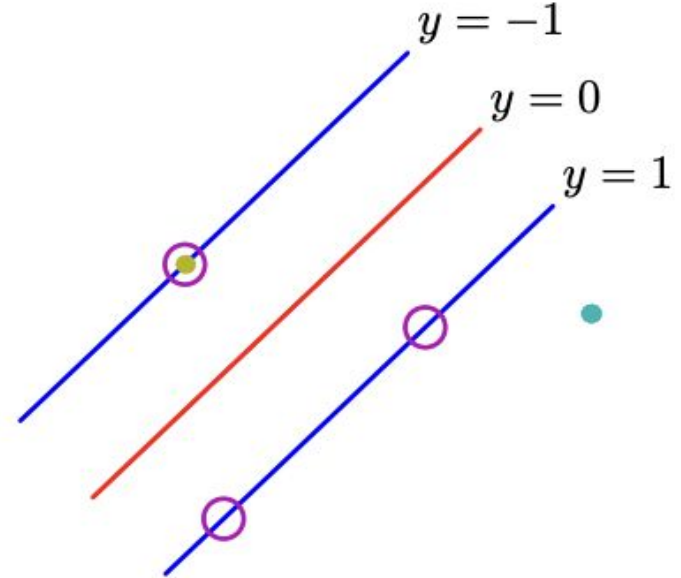
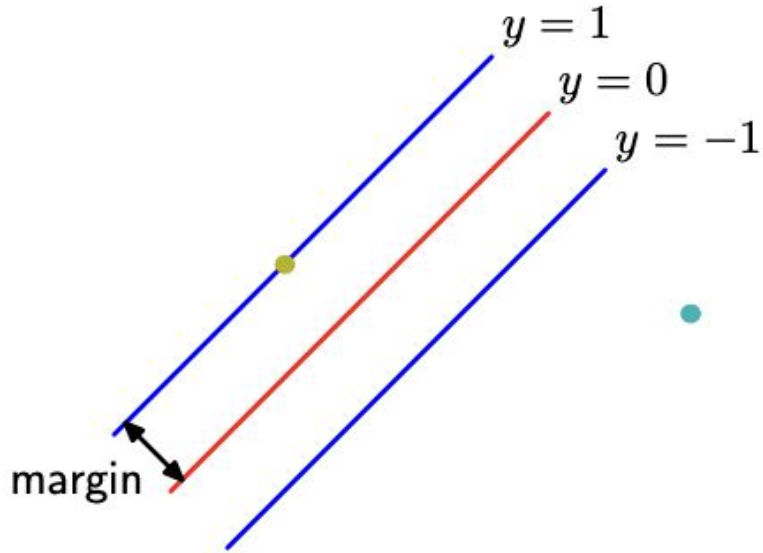
Large Margin

Support Vectors

Margins



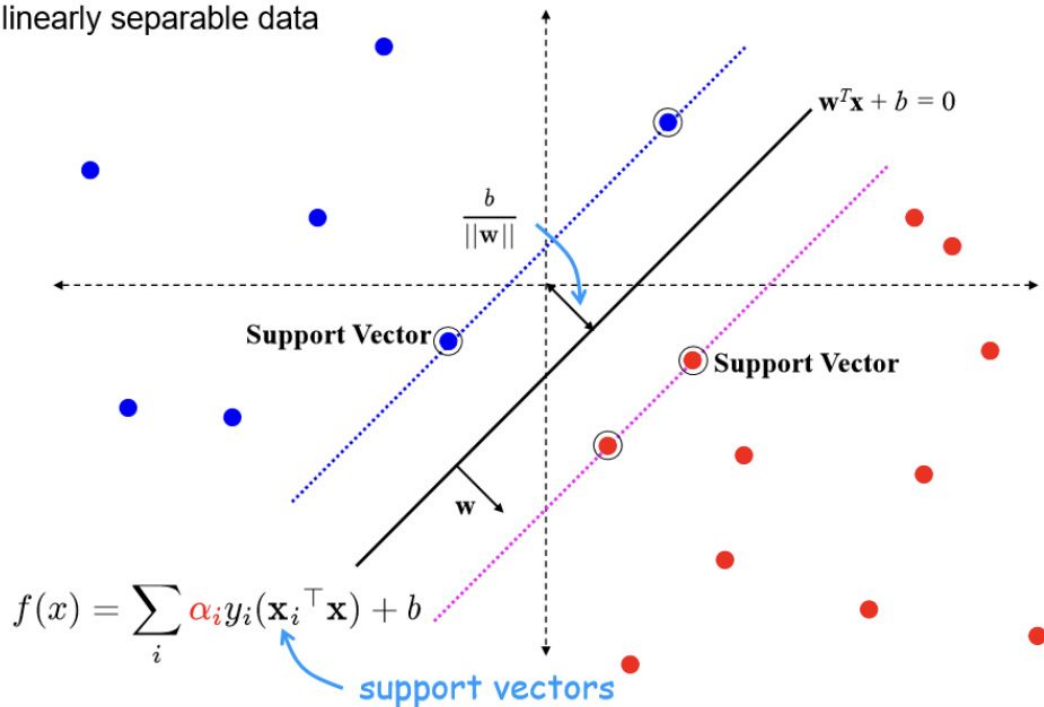
Maximum Margin Classifiers



The margin is defined as the perpendicular distance between the decision boundary and the closest of the data points, as shown on the left figure. Maximizing the margin leads to a particular choice of decision boundary, as shown on the right. The location of this boundary is determined by a subset of the data points, known as **support vectors**, which are indicated by the circles.

Support Vector Machines

linearly separable data



Hard margin optimization

- Find $\mathbf{w}$ and b such that

$$\max_{\mathbf{w}} \frac{2}{\|\mathbf{w}\|} \quad \text{subject to } \mathbf{w}^\top \mathbf{x}_i + b \begin{cases} \geq 1 & \text{if } y_i = +1 \\ \leq -1 & \text{if } y_i = -1 \end{cases} \quad \text{for } i = 1 \dots N$$

- Which is the same as:

Find $\mathbf{w}$ and b such that

$$\min_{\mathbf{w}} \|\mathbf{w}\|^2 \quad \text{subject to } y_i (\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \text{for } i = 1 \dots N$$

Where, $\|\mathbf{w}\|^2 = \mathbf{w}^\top \mathbf{w}$

- Need to optimize a *quadratic* function subject to *linear* constraints.

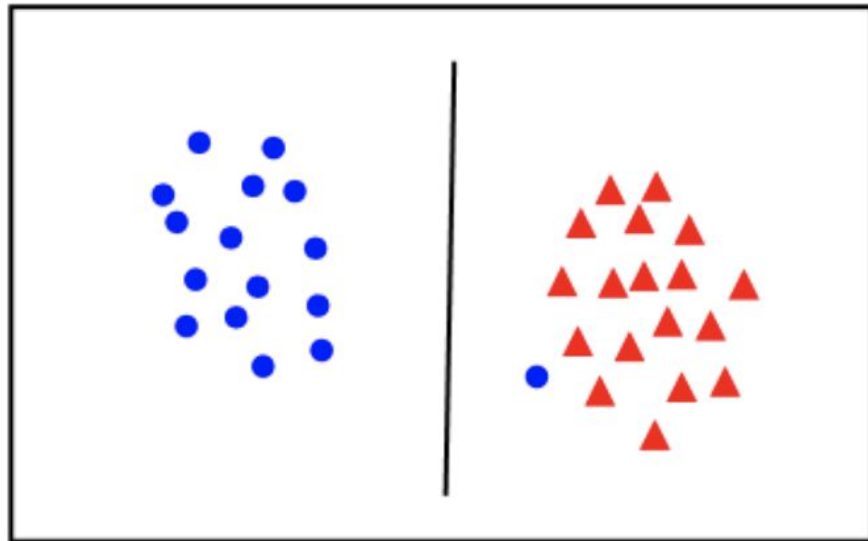
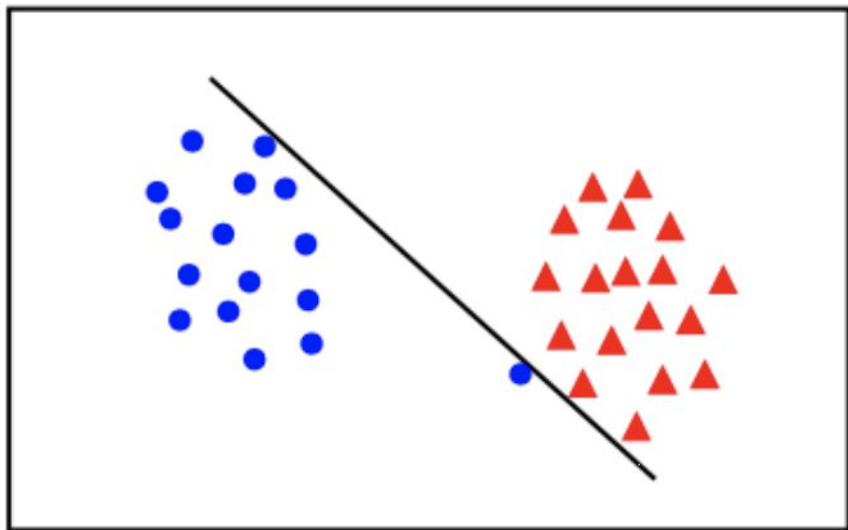
Overlapping Class Distributions

Main Algorithm:

So far, we have assumed that the training data points are linearly separable. In practice, however, the class-conditional distributions may overlap, in which case exact separation of the training data can lead to poor generalization. We therefore need a way to modify the support vector machine so as to allow some of the training points to be misclassified.

In the case of separable classes, we implicitly used an error function that gave **infinite error** if a data point was misclassified and **zero error** if it was classified correctly, and then optimized the model parameters to **maximize the margin**. We now modify this approach so that data points are allowed to be on the 'wrong side' of the margin boundary, but with a penalty that increases with the distance from that boundary. For the subsequent optimization problem, it is convenient to make this penalty a linear function of this distance.

Overlapping Class Distributions



Overlapping Class Distributions

Main Algorithm:

To do this, we introduce slack variables, $\xi_n \geq 0$ where $n = 1, \dots, N$, with one slack variable for each training data point. These are defined by $\xi_n = 0$ for data points that are on or inside the correct margin boundary and $\xi_n = |t_n - y(x_n)|$ for other points. Thus a data point that is on the decision boundary $y(x_n)=0$ will have $\xi_n = 1$, and points with $\xi_n > 1$ will be misclassified. The exact classification constraints are then replaced with:

$$t_n y(\mathbf{x}_n) \geq 1 - \xi_n, \quad n = 1, \dots, N$$

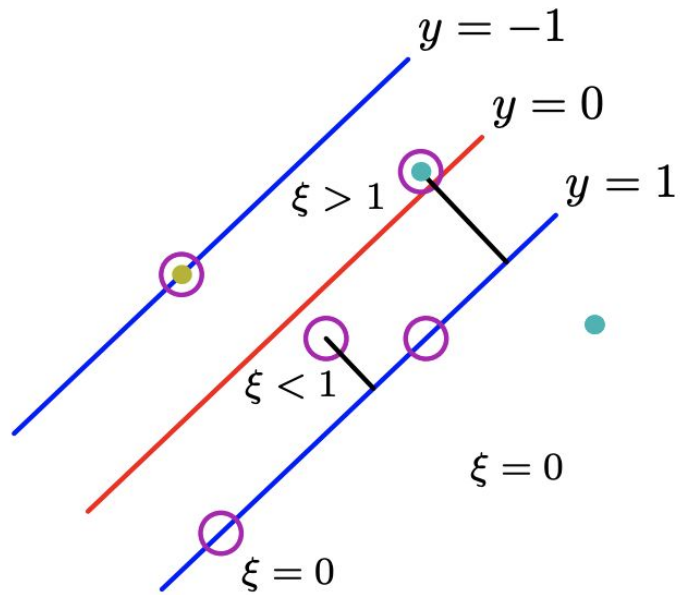
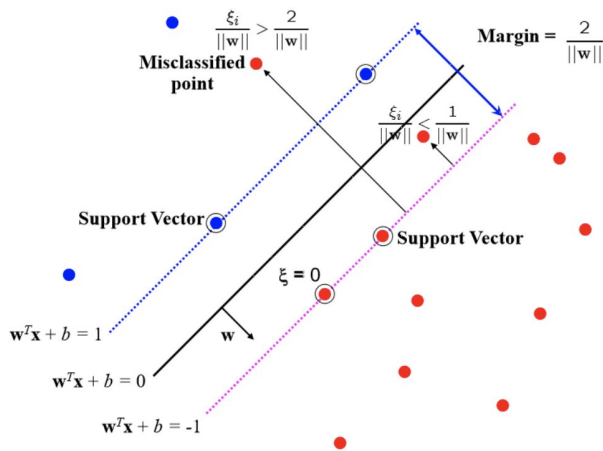
in which the slack variables are constrained to satisfy $\xi_n \geq 0$.

Data points for which

- $\xi_n = 0$ are correctly classified and are either on the margin or on the correct side of the margin.
- Points for which $0 < \xi_n \leq 1$ lie inside the margin, but on the correct side of the decision boundary.
- When $\xi_n > 1$ lie on the wrong side of the decision boundary and are misclassified

Overlapping Class Distributions

This is sometimes described as relaxing the hard margin constraint to give a soft margin and allows some of the training set data points to be misclassified. Note that while slack variables allow for overlapping class distributions, this framework is still sensitive to outliers because the penalty for misclassification increases linearly with ξ .



Overlapping Class Distributions

Main Algorithm:

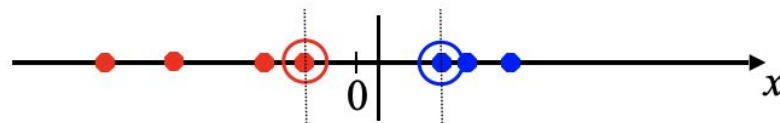
Our goal is now to maximize the margin while softly penalizing points that lie on the wrong side of the margin boundary. We therefore minimize:

$$C \sum_{n=1}^N \xi_n + \frac{1}{2} \|\mathbf{w}\|^2$$

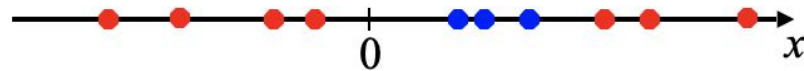
where the parameter $C > 0$ controls the trade-off between the slack variable penalty and the margin. Because any point that is misclassified has $\xi_n > 1$, it follows that $\sum_n \xi_n$ is an upper bound on the number of misclassified points. The parameter C is therefore analogous to (the inverse of) a regularization coefficient because it controls the trade-off between minimizing training errors and controlling model complexity. In the limit $C \rightarrow \infty$, we will recover the earlier support vector machine for separable data.

Non-linear SVMs

- Some datasets are linearly separable with some noise

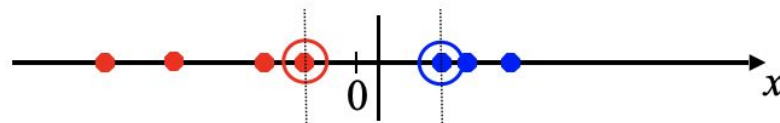


- Sometimes not linearly separable

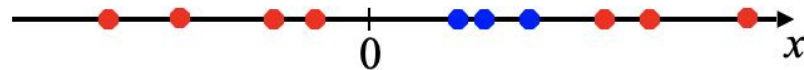


Non-linear SVMs

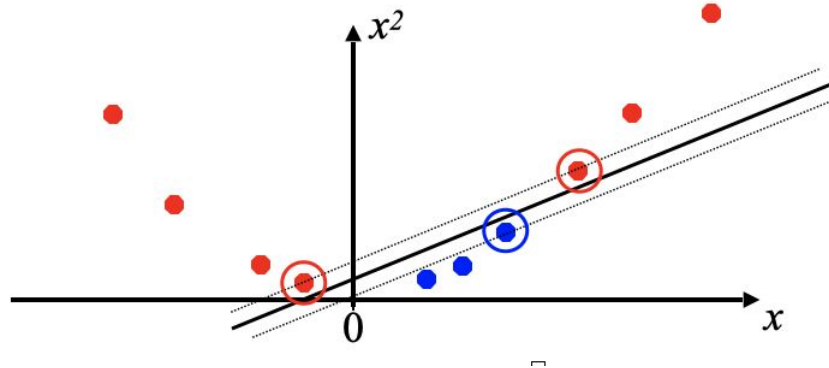
- Some datasets are linearly separable with some noise



- Sometimes not linearly separable

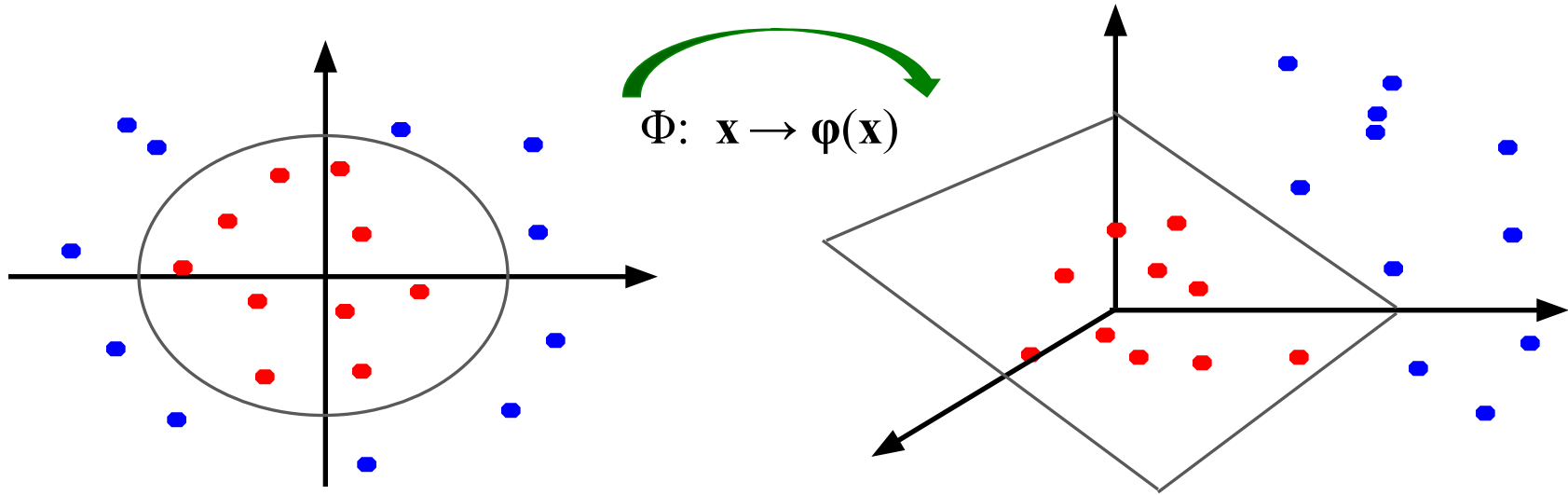


- Map the data onto a higher dimension



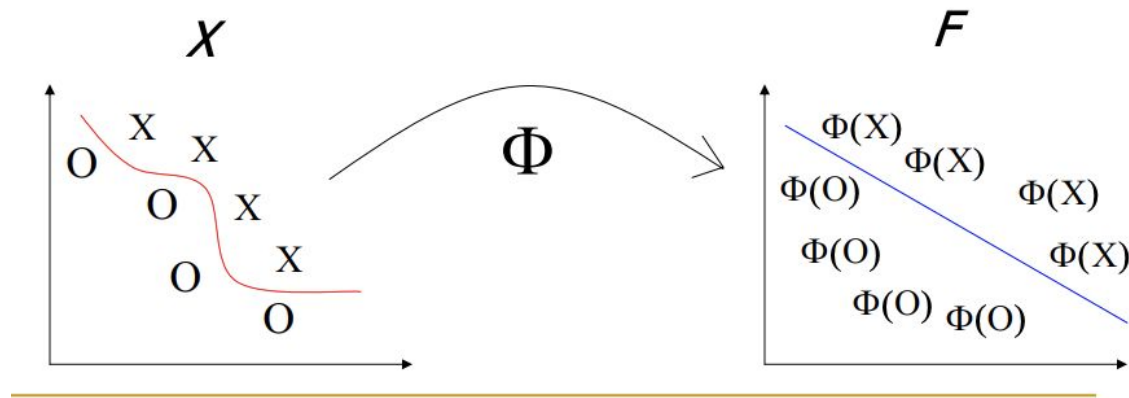
Non-linear SVMs

General idea: the original feature space can always be mapped to some higher-dimensional feature space where the training set is separable:



Kernel trick

Map the data into a (possibly high dimensional) vector space where linear relations exist among the data, then apply a linear algorithm in this space



Kernel trick

- A function that takes as its inputs vectors in the original space and returns the dot product of the vectors in the feature space is called a kernel function
- More formally, we will call K a kernel if there exists $\boldsymbol{\varphi}: \mathbf{x} \rightarrow \boldsymbol{\varphi}(\mathbf{x})$ such that for all $\mathbf{x}, \mathbf{y} \in X$, the inner product becomes:

$$K(\mathbf{x}, \mathbf{y}) = \boldsymbol{\varphi}(\mathbf{x})^T \boldsymbol{\varphi}(\mathbf{y})$$

Kernel trick

- The linear classifier relies on inner product between vectors $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- If every datapoint is mapped into high-dimensional space via some transformation $\Phi: \mathbf{x} \rightarrow \varphi(\mathbf{x})$, the inner product becomes:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j)$$

- Example:
 - 2-dimensional vectors $\mathbf{x} = [x_1 \ x_2]$; let $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^2$,
 - Need to show that $K(\mathbf{x}_i, \mathbf{x}_j) = \varphi(\mathbf{x}_i)^T \varphi(\mathbf{x}_j)$

Kernel trick

- The linear classifier relies on inner product between vectors $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- If every datapoint is mapped into high-dimensional space via some transformation $\Phi: \mathbf{x} \rightarrow \boldsymbol{\varphi}(\mathbf{x})$, the inner product becomes:

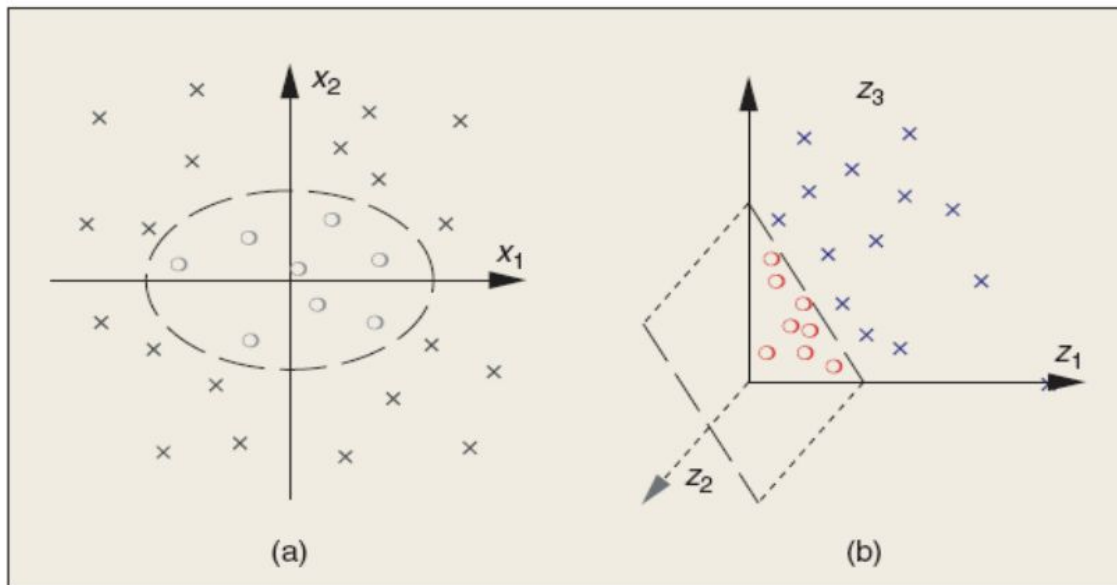
$$K(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\varphi}(\mathbf{x}_i)^T \boldsymbol{\varphi}(\mathbf{x}_j)$$

- Example:
 - 2-dimensional vectors $\mathbf{x} = [x_1 \ x_2]$; let $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^2$,
 - Need to show that $K(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\varphi}(\mathbf{x}_i)^T \boldsymbol{\varphi}(\mathbf{x}_j)$

$$\begin{aligned} K(\mathbf{x}_i, \mathbf{x}_j) &= (1 + \mathbf{x}_i^T \mathbf{x}_j)^2 = 1 + x_{i1}^2 x_{j1}^2 + 2 x_{i1} x_{j1} x_{i2} x_{j2} + x_{i2}^2 x_{j2}^2 + 2 x_{i1} x_{j1} + 2 x_{i2} x_{j2} = \\ &= [1 \ x_{i1}^2 \ \sqrt{2} x_{i1} x_{i2} \ x_{i2}^2 \ \sqrt{2} x_{i1} \ \sqrt{2} x_{i2}]^T [1 \ x_{j1}^2 \ \sqrt{2} x_{j1} x_{j2} \ x_{j2}^2 \ \sqrt{2} x_{j1} \ \sqrt{2} x_{j2}] = \\ &= \boldsymbol{\varphi}(\mathbf{x}_i)^T \boldsymbol{\varphi}(\mathbf{x}_j), \text{ where } \boldsymbol{\varphi}(\mathbf{x}) = [1 \ x_1^2 \ \sqrt{2} x_1 x_2 \ x_2^2 \ \sqrt{2} x_1 \ \sqrt{2} x_2] \end{aligned}$$

- Thus, a kernel function *implicitly* maps data to a high-dimensional space (without the need to compute each $\boldsymbol{\varphi}(\mathbf{x})$ explicitly).

Kernel trick



▲ 1. Effect of the map $\phi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$ (a) Input space $\mathcal{X}$ and (b) feature space $\mathcal{H}$.

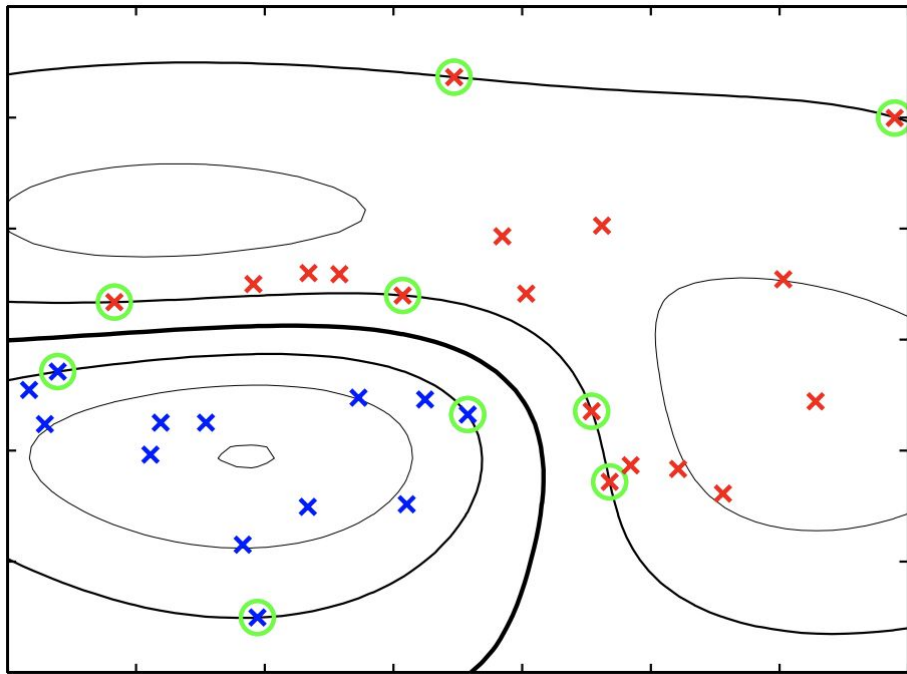
Examples of Kernel functions

- Linear: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- Polynomial of power p : $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^p$
- Gaussian (radial-basis function): $K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}}$

Non-linear SVM Example

Here we have an example of the classification resulting from training a support vector machine on a simple synthetic data set using a Gaussian kernel. Although the data set is not linearly separable in the two-dimensional data space $\mathbf{x}$, it is linearly separable in the nonlinear feature space defined implicitly by the nonlinear kernel function. Thus the training data points are perfectly separated in the original data space.

This example also provides a geometrical insight into the origin of sparsity in the SVM. The maximum margin hyperplane is defined by the location of the support vectors. Other data points can be moved around freely (so long as they remain outside the margin region) without changing the decision boundary, and so the solution will be independent of such data points.



Other links

- [Tensorflow playground](#)